

SMART RECRUIT

Full Technical Stack Document

Project Technical Stack Overview

This document describes the complete technical stack used to develop and run the Smart Recruit recruitment matching platform (ASP.NET Core web application with a static frontend).

Project Name:	Smart Recruit / Recruitment-Project
Document Type:	Full Technical Stack Document
Date:	11 August 2026
Target Framework:	.NET 8.0 (net8.0)

1. .NET Version Used in the Project

The project targets .NET 8.0 using the Microsoft.NET.Sdk.Web SDK.

Target Framework:	net8.0
SDK / Runtime family:	.NET 8 (ASP.NET Core 8)
Observed SDK locally:	.NET SDK 10.0.302 (compatible for building net8.0)
- Project file: Recruitment-Project.csproj - Nullable reference types enabled - Implicit usings enabled	

2. Frontend Technology / Framework

The frontend is a static multi-page application hosted from ASP.NET Core wwwroot. It does not use React, Angular, or Vue.

- HTML5 pages for Job Seeker, Employer, and Administrator modules
- Vanilla JavaScript (modular scripts under wwwroot/shared and wwwroot/module*)
- CSS3 for layout, responsive design, and UI styling
- Fetch API for REST calls to the backend
- Browser localStorage for JWT session (token, user, expiration)

Main frontend structure:

- wwwroot/index.html, login.html, register.html
- wwwroot/module1-core - Admin UI
- wwwroot/module2-jobseeker - Job Seeker UI
- wwwroot/module3-employer - Employer UI

- wwwroot/module4-applications - Applications / interviews / contact requests
- wwwroot/module5-trust - Reports, notifications, moderation audit
- wwwroot/shared - apiClient.js, auth.js, guards.js, ui.js, utils.js, common.css

3. Backend Technology / Framework

The backend is built with ASP.NET Core Web API (MVC Controllers) on .NET 8.

- ASP.NET Core 8 Web Application
- RESTful API Controllers under Controllers/
- Service layer under Services/
- Repository layer under Repositories/
- DTOs under DTOs/
- Entity models and enums under Models/
- Custom middleware: ExceptionMiddleware, ActiveAccountMiddleware
- Dependency Injection via built-in ASP.NET Core DI container
- Static file hosting for the frontend (UseDefaultFiles / UseStaticFiles)

4. Database Used

Microsoft SQL Server is used through Entity Framework Core.

Database Engine:	Microsoft SQL Server (LocalDB for local development)
LocalDB Instance:	(localdb)\MSSQLLocalDB
Database Name:	RecruitmentProjectDb
ORM:	Entity Framework Core 8.0.8
Provider:	Microsoft.EntityFrameworkCore.SqlServer 8.0.8

- Code-First approach with EF Core Migrations
- DbContext: AppDbContext (Data/AppDbContext.cs)
- Connection configured in appsettings.json (ConnectionStrings:DefaultConnection)

5. Programming Languages

- C# - Backend API, business logic, data access, middleware
- JavaScript (ES6+) - Frontend application logic and API integration
- HTML5 - Page structure and forms
- CSS3 - Styling and responsive layout
- SQL - Generated/executed by Entity Framework Core against SQL Server
- JSON - Configuration (appsettings) and API request/response payloads

6. APIs / Services Used

The system exposes a custom REST API under /api/*. Swagger/OpenAPI is enabled in Development.

Key API areas:

- Auth API - register job seeker/employer, login, current user (/api/Auth)
- Job Seeker API - profile, dashboard, CV (/api/jobseekers)
- Jobs / Matching API - job search and details with match scores (/api/jobs)
- Employer API - company profile and dashboard (/api/employers)
- Employer Vacancies API - create/update/submit/close (/api/employer/vacancies)
- Employer Verification API - verification submit/documents
- Applications API - apply, list, update status
- Interview Schedules API - schedule and list interviews
- Contact Requests API - employer requests and job seeker responses
- Notifications API - list, unread count, mark read
- Job Reports API - job seeker reports and admin review
- Admin API - users, dashboard, vacancy approval, verification review, moderation

Internal application services include AuthService, JwtTokenService, JobSeekerService, MatchingService, JobSearchService, EmployerService, VacancyService, ApplicationService, InterviewScheduleService, ContactRequestService, NotificationService, JobReportService, ModerationService, CvFileStorageService, VerificationDocumentStorageService, and CacheService.

Integrated protocols / platform services:

- JWT Bearer Authentication (Microsoft.AspNetCore.Authentication.JwtBearer)
- OpenAPI / Swagger UI (Swashbuckle.AspNetCore) for API documentation and testing
- In-memory caching (IMemoryCache) for selected cache operations

7. Other Tools, Libraries, or Technologies Used

NuGet packages / libraries:

- BCrypt.Net-Next 4.0.3 - password hashing and verification
- Microsoft.AspNetCore.Authentication.JwtBearer 8.0.8 - JWT authentication
- Microsoft.EntityFrameworkCore 8.0.8 - ORM core
- Microsoft.EntityFrameworkCore.SqlServer 8.0.8 - SQL Server provider
- Microsoft.EntityFrameworkCore.Design / Tools 8.0.8 - migrations tooling
- Microsoft.OpenApi 1.6.22 - OpenAPI model support
- Swashbuckle.AspNetCore 6.6.2 - Swagger generation and UI

Architecture and engineering practices:

- Layered architecture: Controllers -> Services -> Repositories -> EF Core / SQL Server
- Interface-based dependency injection
- Role-based authorization (Administrator, Employer, JobSeeker)
- File storage for CV uploads and employer verification documents
- Deterministic matching scores (skills, experience, education, location)
- Trust and moderation features (reports, warn/suspend/disable, audit)

Development and tooling environment:

- Visual Studio / Visual Studio Code / Cursor IDE
- Git for source control
- SQL Server LocalDB for local database development
- Swagger UI at /swagger during Development
- launchSettings.json profiles for HTTP/HTTPS local hosting

End of Document - This technical stack summary is based on the current Smart Recruit (Recruitment-Project) codebase configuration and package references.